

**School of Computer Science and Artificial
Intelligence**

WEEK-10 Assignment # 19

Program : B. Tech (CSE)
Specialization :AIML
Course Title : Competitive Programming
Course Code : 23CS302PC305
Semester : VI
Academic Session : 2025-2026
Name of Student : B.Thanmai
Enrollment No. : 2303A52126
Batch No. : 33
Date :24/03/26

Primality Testing — Miller-Rabin Primality Test

Context

A quick-check service needs to determine if large numbers are prime with probabilistic certainty.

Task

- Implement a probabilistic primality test using the Miller-Rabin algorithm.
- Allow a configurable number of bases to trade accuracy for speed. For 64-bit integers, use deterministic bases to guarantee correctness (known sets exist).

Requirements

Input:

n (integer to test) t trials
(number of random bases to test;

optional) Output: "prime" or

"composite" Edge cases:

$n < 2$ (not prime) Small
primes (2, 3)

Sample

Input:

97 5 Output:

prime

Implementation tips

- Decompose $n-1 = d * 2^s$ with d odd.
- For each base a , compute $a^d \bmod n$, then square s times checking witnesses.
- Use deterministic bases for 64-bit: e.g., {2,3,5,7,11,13,17} or the widely cited sets for 64-bit.

PYTHON:

```
File Edit Selection View Go ... Week10
Main.py X
1 import random
2
3 def miller_rabin(n, trials=5):
4     # Edge cases
5     if n < 2:
6         return "composite"
7     if n in (2, 3):
8         return "prime"
9     if n % 2 == 0:
10        return "composite"
11
12    # Write n-1 = d * 2^s
13    d = n - 1
14    s = 0
15    while d % 2 == 0:
16        d //= 2
17        s += 1
18
19    # Deterministic bases for 64-bit integers
20    if n < 2**64:
21        bases = [2, 3, 5, 7, 11, 13, 17]
22    else:
23        bases = [random.randrange(2, n - 1) for _ in range(trials)]
24
25    for a in bases:
26        if a >= n:
27            continue
28
29        x = pow(a, d, n)
30
31        if x == 1 or x == n - 1:
32            continue
33
34        for _ in range(s - 1):
35            x = pow(x, 2, n)
36            if x == n - 1:
37                break
38        else:
39            return "composite"
40
41    return "prime"
42
43 # Input
44 n, trials = map(int, input().split())
45
46 print(miller_rabin(n, trials))
Ln 47, Col 31 Spaces: 4 UTF-8 CRLF Python 3.11.9 (Microsoft Store) Go Live
```

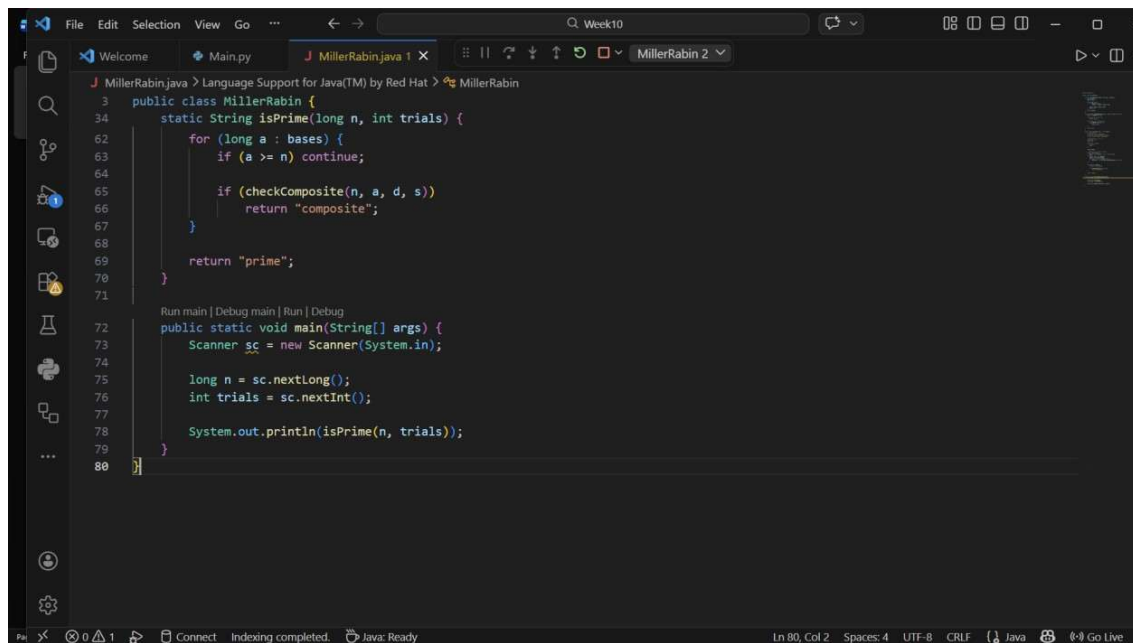
```
File Edit Selection View Go ... Week10
Main.py X
3 def miller_rabin(n, trials=5):
31     if x == 1 or x == n - 1:
32         continue
33
34     for _ in range(s - 1):
35         x = pow(x, 2, n)
36         if x == n - 1:
37             break
38     else:
39         return "composite"
40
41     return "prime"
42
43 # Input
44 n, trials = map(int, input().split())
45
46 print(miller_rabin(n, trials))
Ln 47, Col 31 Spaces: 4 UTF-8 CRLF Python 3.11.9 (Microsoft Store) Go Live

python3.11.exe "s:/3 2 sem/CP/Week10/Main.py"
95 7
composite
PS S:\3 2 sem\CP\Week10> & C:/Users/sowmy/AppData/Local/Microsoft/WindowsApps/python3.11.exe "s:/3 2 sem/CP/Week10/Main.py"
97 5
prime
```

JAVA:

```
File Edit Selection View Go ... Week10
J MillerRabin.java 1 X MillerRabin 2
J MillerRabin.java > Language Support for Java(TM) by Red Hat > MillerRabin
1 import java.util.*;
2
3 public class MillerRabin {
4
5     static long modPow(long base, long exp, long mod) {
6         long result = 1;
7         base %= mod;
8
9         while (exp > 0) {
10             if ((exp & 1) == 1)
11                 result = (result * base) % mod;
12
13             base = (base * base) % mod;
14             exp >>= 1;
15         }
16         return result;
17     }
18
19     static boolean checkComposite(long n, long a, long d, int s) {
20         long x = modPow(a, d, n);
21
22         if (x == 1 || x == n - 1)
23             return false;
24
25         for (int i = 1; i < s; i++) {
26             x = modPow(x, d, n);
27             if (x == n - 1)
28                 return false;
29         }
30
31         return true;
32     }
33 }
```

```
File Edit Selection View Go ... Week10
J MillerRabin.java 1 X MillerRabin 2
J MillerRabin.java > Language Support for Java(TM) by Red Hat > MillerRabin
3 public class MillerRabin {
4
5     static String isPrime(long n, int trials) {
6         // Edge cases
7         if (n < 2) return "composite";
8         if (n == 2 || n == 3) return "prime";
9         if (n % 2 == 0) return "composite";
10
11         // Decompose n-1 = d * 2^s
12         long d = n - 1;
13         int s = 0;
14
15         while (d % 2 == 0) {
16             d /= 2;
17             s++;
18         }
19
20         long[] bases;
21
22         // Deterministic bases for 64-bit
23         if (n < (1L << 62)) {
24             bases = new long[]{2, 3, 5, 7, 11, 13, 17};
25         } else {
26             Random rand = new Random();
27             bases = new long[trials];
28             for (int i = 0; i < trials; i++) {
29                 bases[i] = 2 + Math.abs(rand.nextLong()) % (n - 3);
30             }
31         }
32
33         for (long a : bases) {
34
35         }
36     }
37 }
```



```
1 public class MillerRabin {
2     static String isPrime(long n, int trials) {
3         for (long a : bases) {
4             if (a >= n) continue;
5             if (checkComposite(n, a, d, s))
6                 return "composite";
7         }
8         return "prime";
9     }
10
11     public static void main(String[] args) {
12         Scanner sc = new Scanner(System.in);
13
14         long n = sc.nextLong();
15         int trials = sc.nextInt();
16
17         System.out.println(isPrime(n, trials));
18     }
19 }
```

```
PS S:\3 2 sem\CP\Week10> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\sowmy\AppData\Roaming\Code\User\workspaceStorage\76cf7d019190988802a9b2b90ef889fa\redhat.java\jdt_ws\Week10_d710793e\bin' 'MillerRabin'
97 5
prime
PS S:\3 2 sem\CP\Week10> 
```